



GPU体系结构

谢旻晖



中國人民大學
RENMIN UNIVERSITY OF CHINA



目录

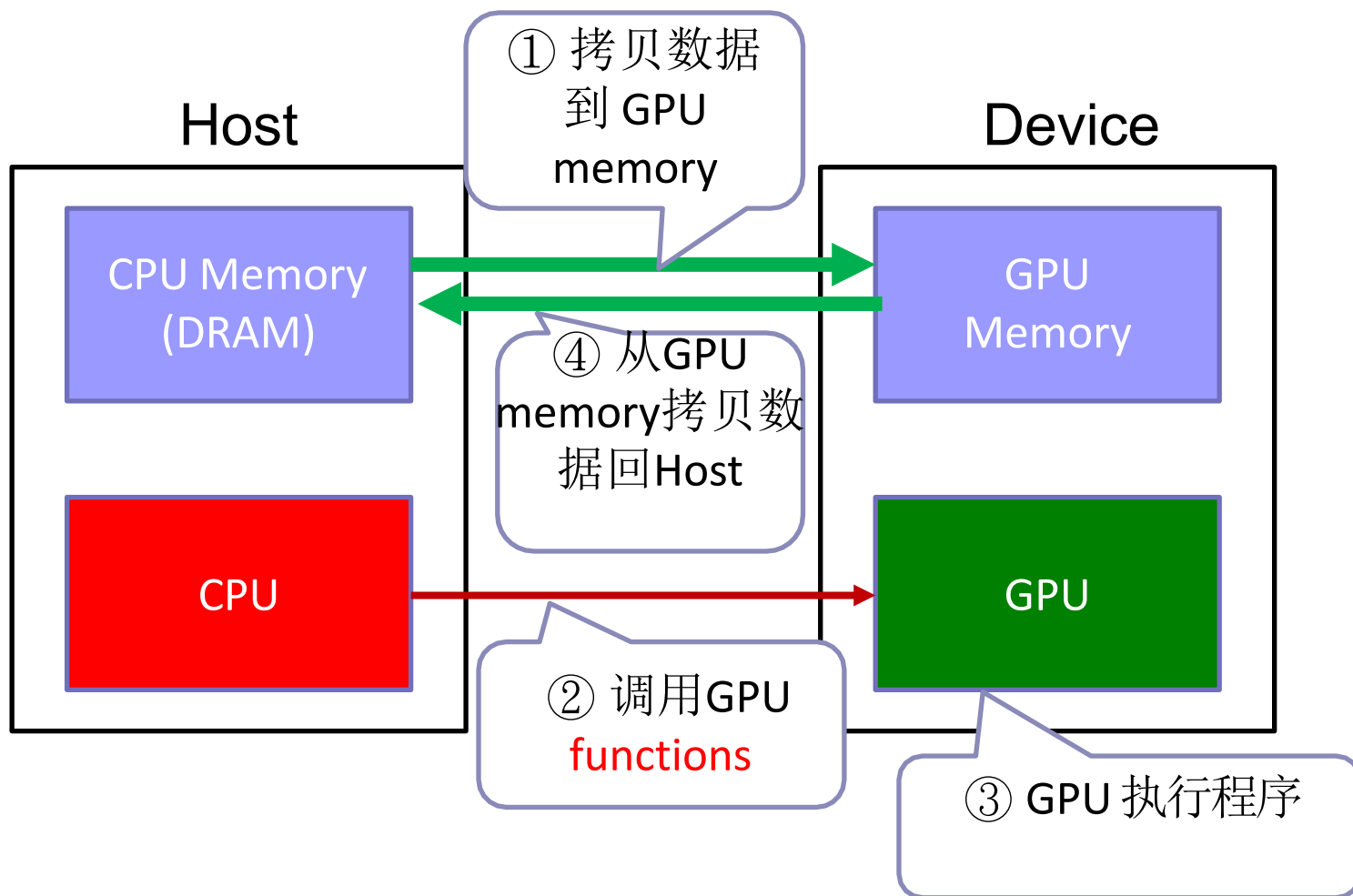
- CUDA内容总结
- 线程执行模型
- 存储层次结构



CUDA基本概念回顾

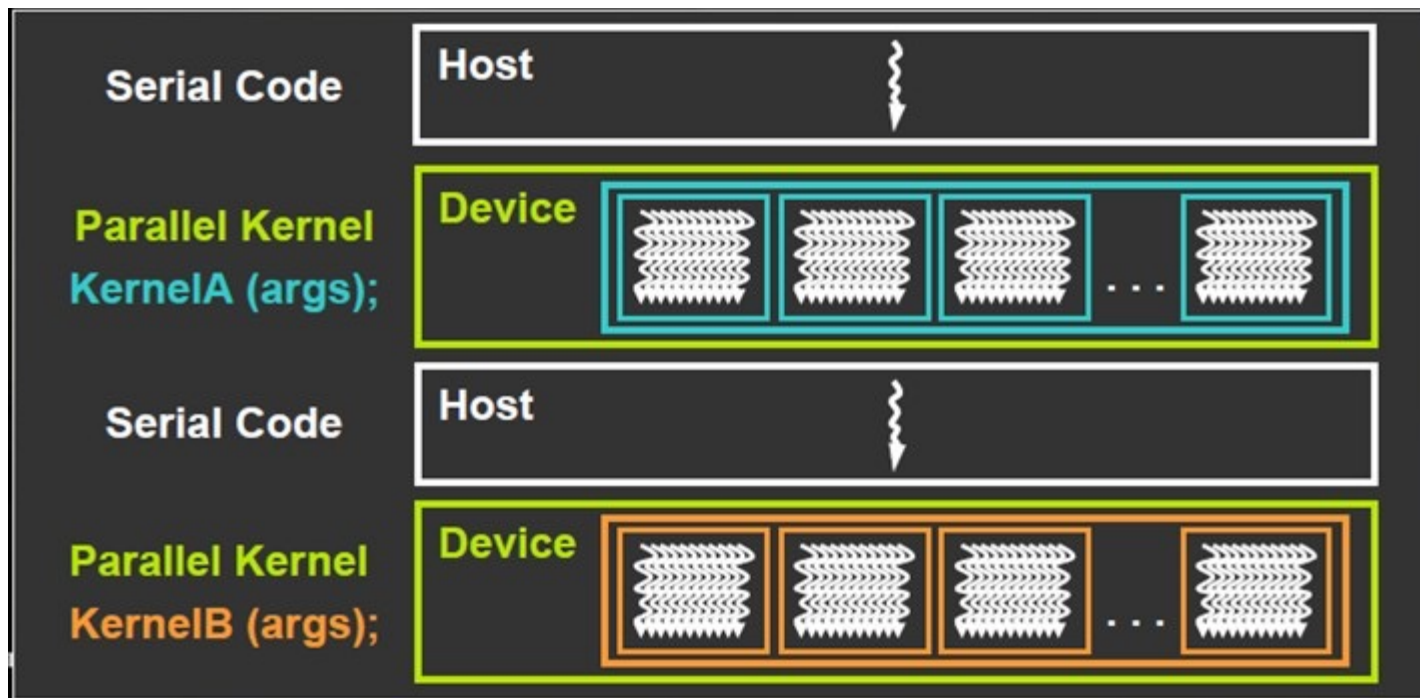
- CUDA: Compute Unified Device Architecture
- CUDA是NVIDIA GPU上的编程工具套件
- 让用户能够使用GPU强大的计算能力
- 对硬件进行了高度抽象
- SIMD方式并行

CUDA程序流程



CUDA编程模型

- CUDA = 串行程序 + 并行kernel
- host线程执行串行C代码
- 并行kernel由多个GPU 线程执行，运行在多个处理单元processing elements上





CUDA编程框架

GPU 代码
(parallel)

CPU 代码
(serial or
parallel if
p-thread/
OpenMP/T
BB/MPI is
used.)

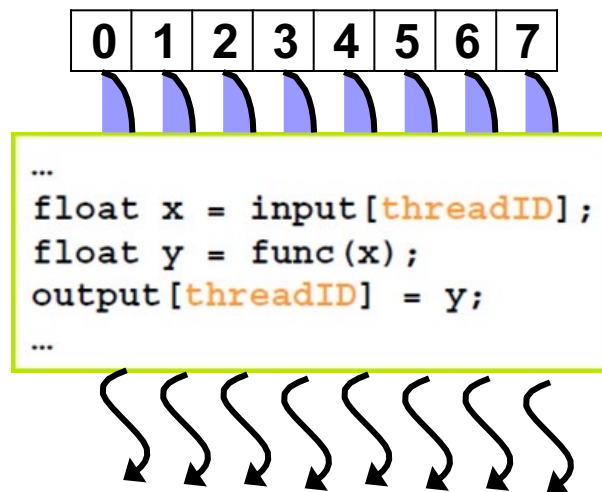
```
#include <cuda_runtime.h>

__global__ void my_kernel(...) {
    ...
}

int main() {
    ...
    cudaMalloc(...)
    cudaMemcpy(...)
    ...
    my_kernel<<<nblock,blocksize>>>>(...)
    ...
    cudaMemcpy(...)
    ...
}
```

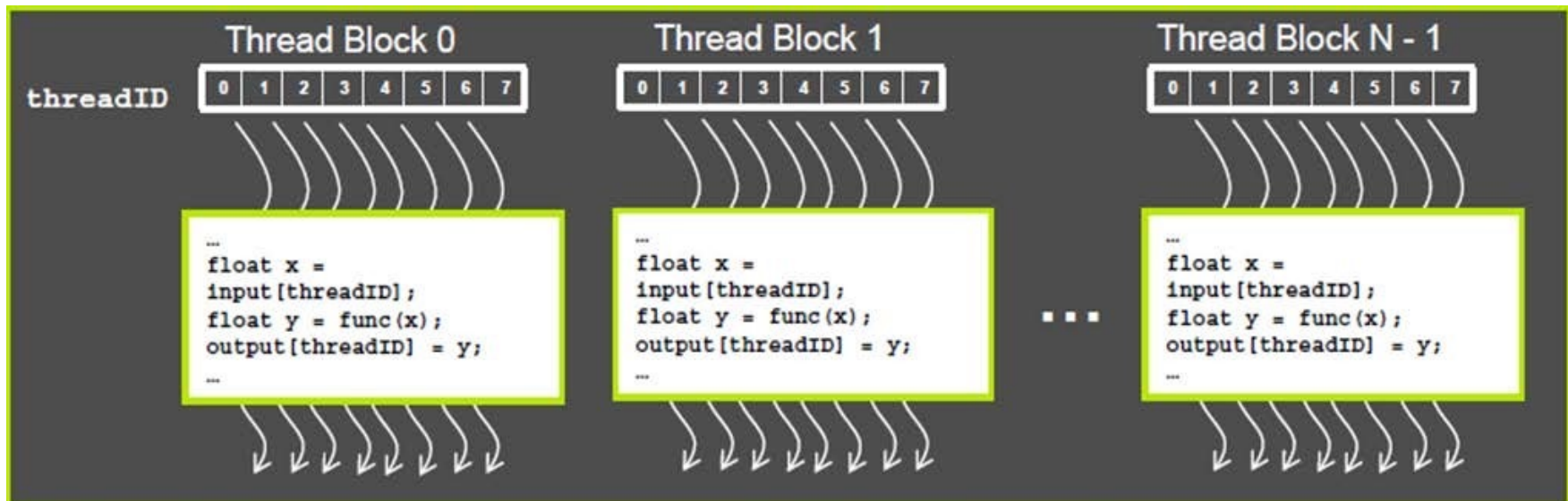
kernel=多个共同运行的线程

- 每个kernel可以有多个线程
 - 每个线程执行同样的code
 - 每个线程根据threadID处理不同的数据



共同运行的线程的层级结构

- 线程被组织成了不同的thread block
- kernel = grid of thread blocks

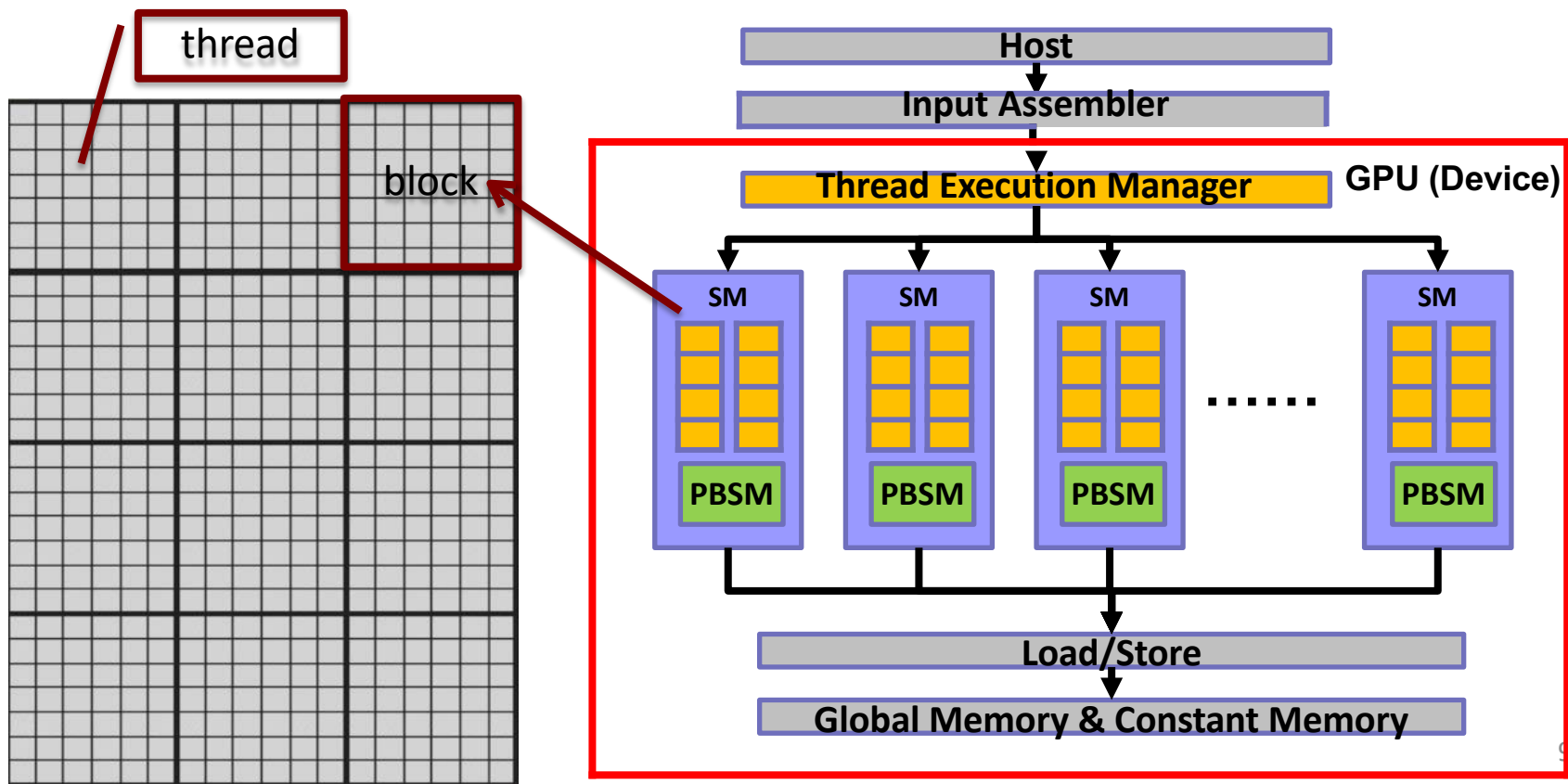


- 同一个block内的线程可以同步

```
scratch[threadID] = begin[threadID];  
__syncthreads();  
int left = scratch[threadID - 1];
```

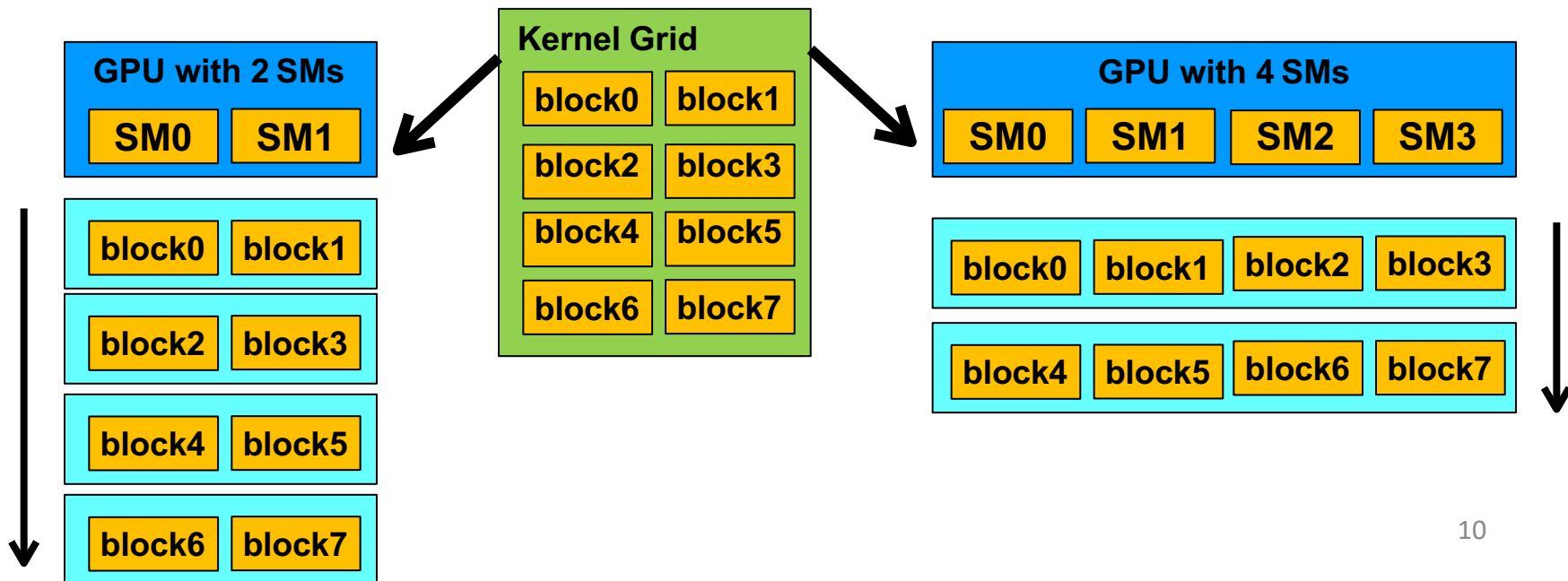
软硬件映射关系

- 软件: grid -> block -> threads
- 硬件: GPU(device) -> SM (multicore processor) -> core



扩展性上的好处

- thread block间不同步
 - 因此，blocks可以以任何顺序并行或串行执行
- 这种独立设计保证了较好的扩展性
 - kernel能够扩展到不同数量的parallel cores中





thread group限制

- 每个block内线程的数目限制
 - 512 before CUDA 2.0
 - 1024 after CUDA 2.0
- 最大block数目的限制
 - 65535 before CUDA 3
 - $2^{31}-1$ after CUDA 3
- 总线程数 = threads/block * block数目
- 使用 `deviceQuery.cpp` 查看当前系统的限制

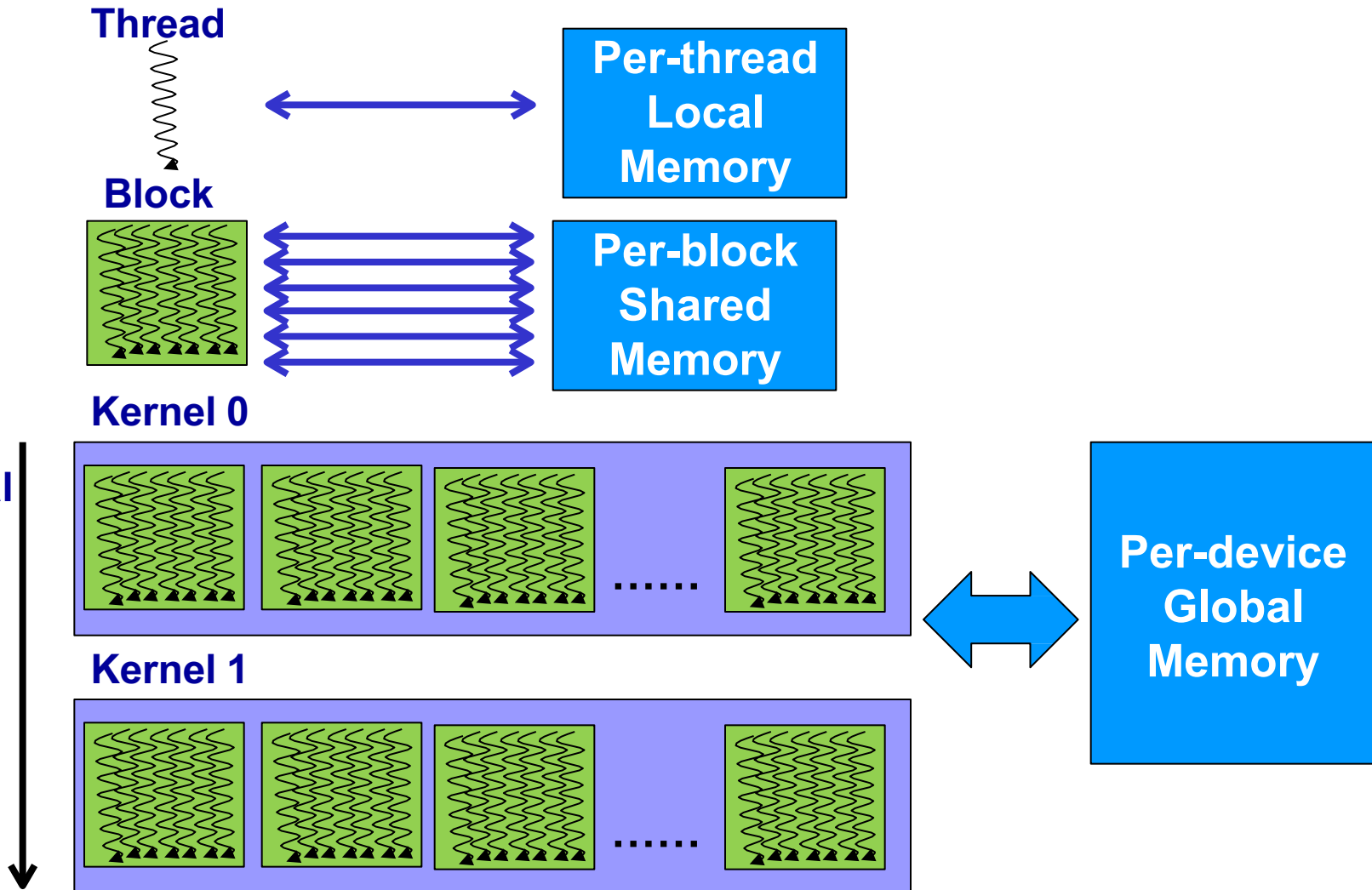


thread group限制

Table 15. Technical Specifications per Compute Capability

	Compute Capability												
Technical Specifications	3.5	3.7	5.0	5.2	5.3	6.0	6.1	6.2	7.0	7.2	7.5	8.0	8.6
Maximum number of resident grids per device (Concurrent Kernel Execution)	32				16	128	32	16	128	16	128		
Maximum dimensionality of grid of thread blocks	3												
Maximum x-dimension of a grid of thread blocks	2 ³¹⁻¹												
Maximum y- or z-dimension of a grid of thread blocks	65535												
Maximum dimensionality of a thread block	3												
Maximum x- or y-dimension of a block	1024												
Maximum z-dimension of a block	64												
Maximum number of threads per block	1024												
Warp size	32												
Maximum number of resident blocks per SM	16		32								16	32	16
Maximum number of resident warps per SM	64										32	64	48
Maximum number of resident threads per SM	2048										1024	2048	1536
Number of 32-bit registers per SM	64 K	128 K	64 K										
Maximum number of 32-bit registers per thread block	64 K				32 K	64 K			32 K	64 K			
Maximum number of 32-bit registers per thread	255												
Maximum amount of shared memory per SM	48 KB	112 KB	64 KB	96 KB	64 KB		96 KB	64 KB	96 KB		64 KB	164 KB	100 KB
Maximum amount of shared memory per thread block ³³	48 KB								96 KB	96 KB	64 KB	163 KB	99 KB
Number of shared memory banks	32												
Maximum amount of local memory per thread	512 KB												
Constant memory size	64 KB												
Cache working set per SM for constant memory	8 KB					4 KB	8 KB						
Cache working set per SM for texture memory	Between 12 KB and 48 KB					Between 24 KB and 48 KB			32 - 128 KB		32 or 64 KB	28KB - 192 KB	28KB - 128 KB
Maximum width for a 1D texture reference bound to a CUDA array	65536					131072							
Maximum width for a 1D texture reference bound to linear memory	2 ²⁷					2 ²⁸	2 ²⁷		2 ²⁸	2 ²⁷	2 ²⁸		
Maximum width and number of layers for a 1D layered texture reference	16384 x 2048					32768 x 2048							
Maximum width and height for a 2D texture reference bound to a CUDA array	65536 x 65536					131072 x 65536							
Maximum width and height for a 2D texture reference bound to linear memory	65000 x 65000		65536 x 65536			131072 x 65000							
Maximum width and height for a 2D texture reference bound to a	16384 x 16384					32768 x 32768							

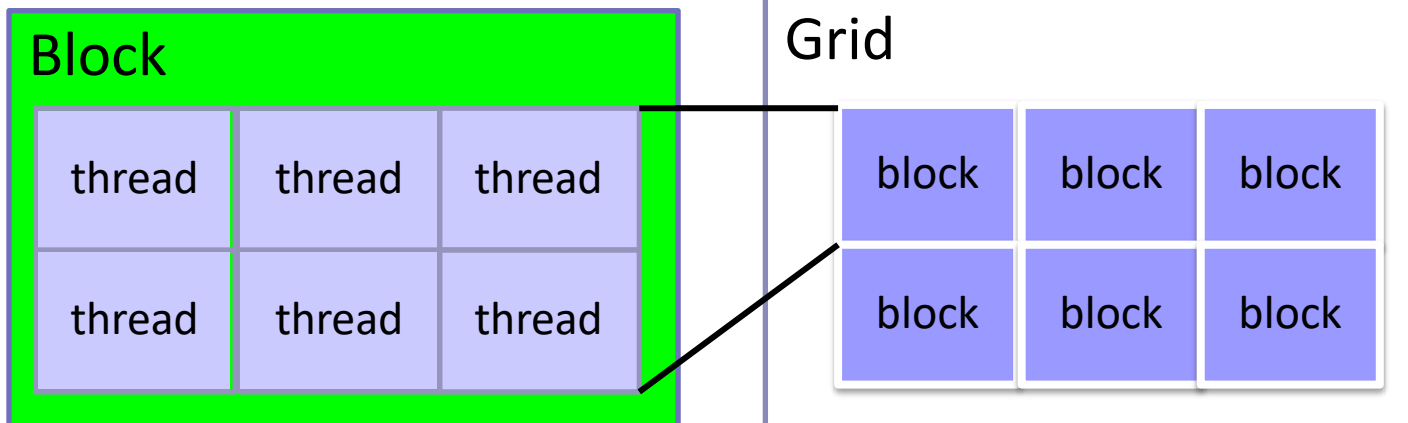
存储结构Memory Hierarchy





CUDA 编程术语

- Host : CPU
- Device : GPU
- Kernel : GPU上运行的function
- Thread : 基本执行单元
- Block : a group of threads
- Grid : a group of blocks





CUDA语言

原则：尽可能使用最少的语句扩展

- Kernel launch

- `kernelFunc<<< nB, nT, nS, Sid >>>(...);` // nS and Sid are optional
 - nB: number of blocks per grid (grid size)
 - nT: number of threads per block (block size)
 - nS: shared memory size (in bytes)
 - Sid: stream ID, default is 0

- 内建的设备变量

- `threadIdx; blockIdx; blockDim; gridDim`

- kernel中可以使用的函数

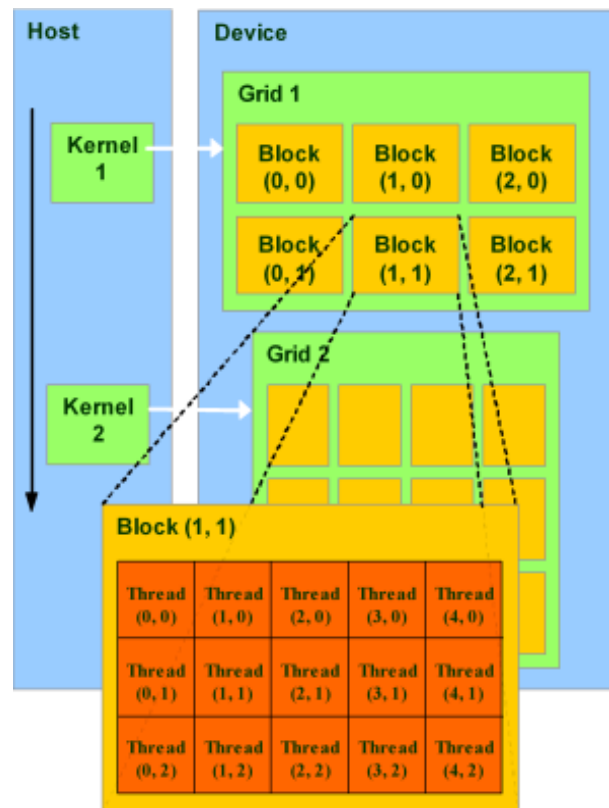
- `__syncthreads();`

- 声明使用位置

<code>__global__ void KernelFunc(...);</code>	// kernel function, run on device
<code>__device__ void GlobalVar;</code>	// variable in device memory
<code>__shared__ void SharedVar;</code>	// variable in per-block shared memory

thread ID和block ID

- 线程ID和block ID是3元组: dim3
- dim3定义在vector_types.h中
- 例子
 - `dim3 grid(3, 2);`
 - `dim3 blk(5, 3);`
 - `my_kernel<<<grid,blk>>>();`
- 每个线程都可以被索引





函数修饰符

函数修饰符	解释
<code>__device__ function</code>	GPU执行 只能由GPU调用
<code>__global__ function</code>	设备执行 只能从host调用 必须return void
<code>__host__ function</code>	host执行 只能从host调用
没有修饰符	host编译
<code>__host__ __device__</code>	同时为CPU和GPU编译



变量修饰符

变量修饰符	解释
<code>__device__ var</code>	保存在GPU的global memory中
<code>__constant__ var</code>	保存在GPU的constant memory中 可以被grid内所有线程访问
<code>__shared__ var</code>	保存在shared memory中 只能由block内线程访问 block销毁后也一起销毁



GPU内存操作

- 三个函数

- `cudaMalloc()`, `cudaFree()`, `cudaMemcpy()`
- Similar to the C's `malloc()`, `free()`, `memcpy()`

1. `cudaMalloc(void **devPtr, size_t size)`

- `devPtr`: 返回分配的GPU内存地址
- `size`: 分配的内存大小 (**bytes**)

2. `cudaFree (void *devPtr)`

3. `cudaMemcpy( void *dst, const void *src, size_t count, enum cudaMemcpyKind kind)`

- `count`: **host**和**device**间内存拷贝操作**bytes**



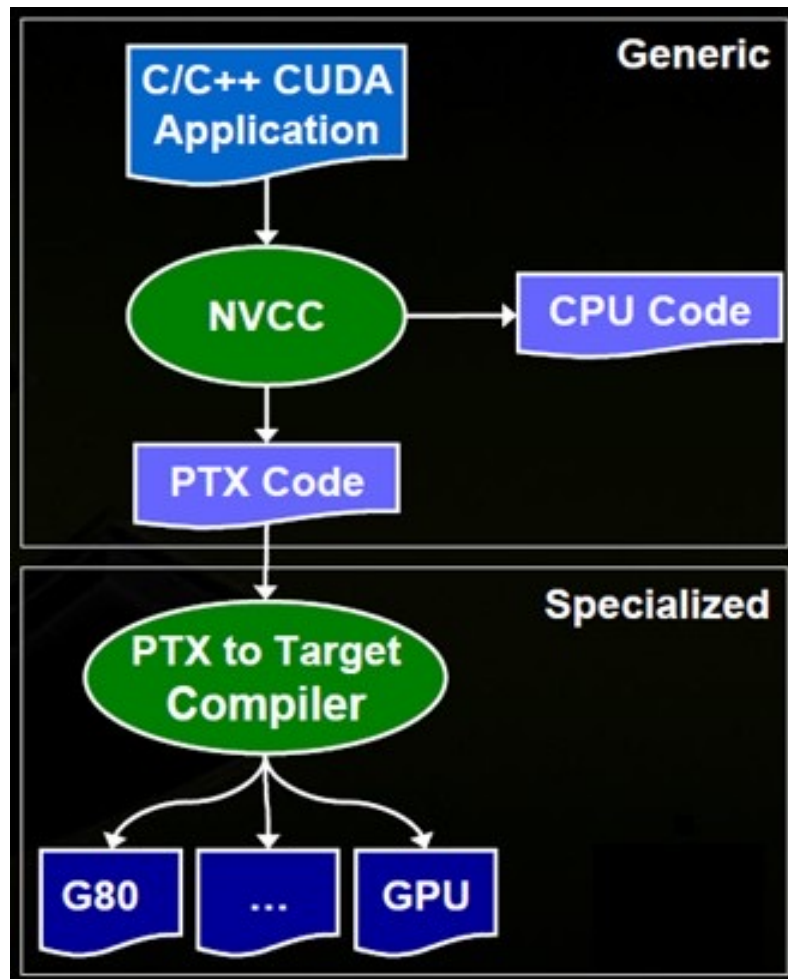
cudaMemcpy类型

cudaMemcpy类型	解释	dst	src
cudaMemcpyHostToHost	Host → Host	host	host
cudaMemcpyHostToDevice	Host → Device	device	host
cudaMemcpyDeviceToHost	Device → Host	host	device
cudaMemcpyDeviceToDevice	Device → Device	device	device

host to host 和 memcpy() 功能一样

程序编译

- 任何包含**CUDA**的代码都必须由**NVCC**编译
 - NVCC会将host和device代码分开
- 两个编译阶段
 - 虚拟ISA
 - PTX: Parallel Thread eXecutions
 - GPU能是别的二进制码





思考



- CUDA软件、硬件映射关系是什么样的？



目录

- CUDA内容总结
- 线程执行模型
 - 执行模型
 - warp
 - warp divergence
- 存储层次结构

执行模型

Software



Thread



Thread block



Grid

Hardware



Scalar processor



SM



GPU device

- 线程由大量的scalar processor执行
- thread block在SM上执行
- 多个thread block可以放在同一个SM上
- kernel由grid of thread blocks开启

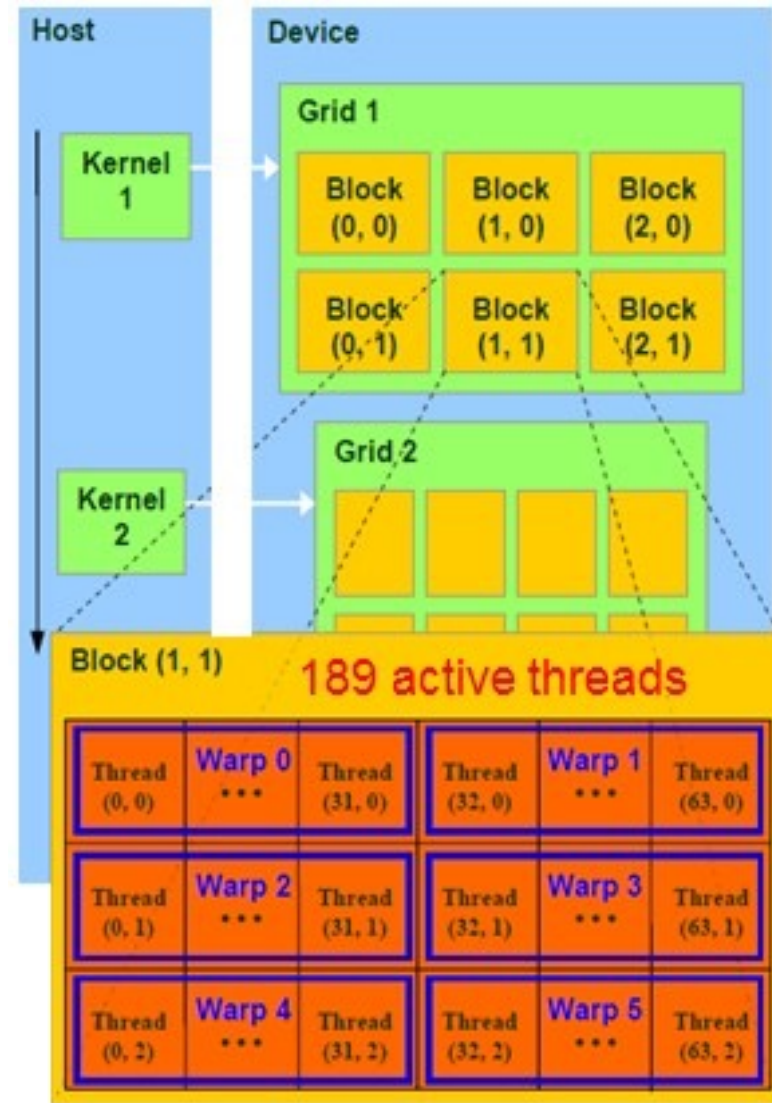


线程执行

- CUDA线程被组织成block
 - 同一个block内的所有线程执行在同一个SM上
 - SM有shared memory, block内的线程可以通信
- 硬件调度器在空闲的SM上调度thread block
 - 不保证block执行的顺序
 - 如果SM有更多的资源, 硬件可以分配更多的block

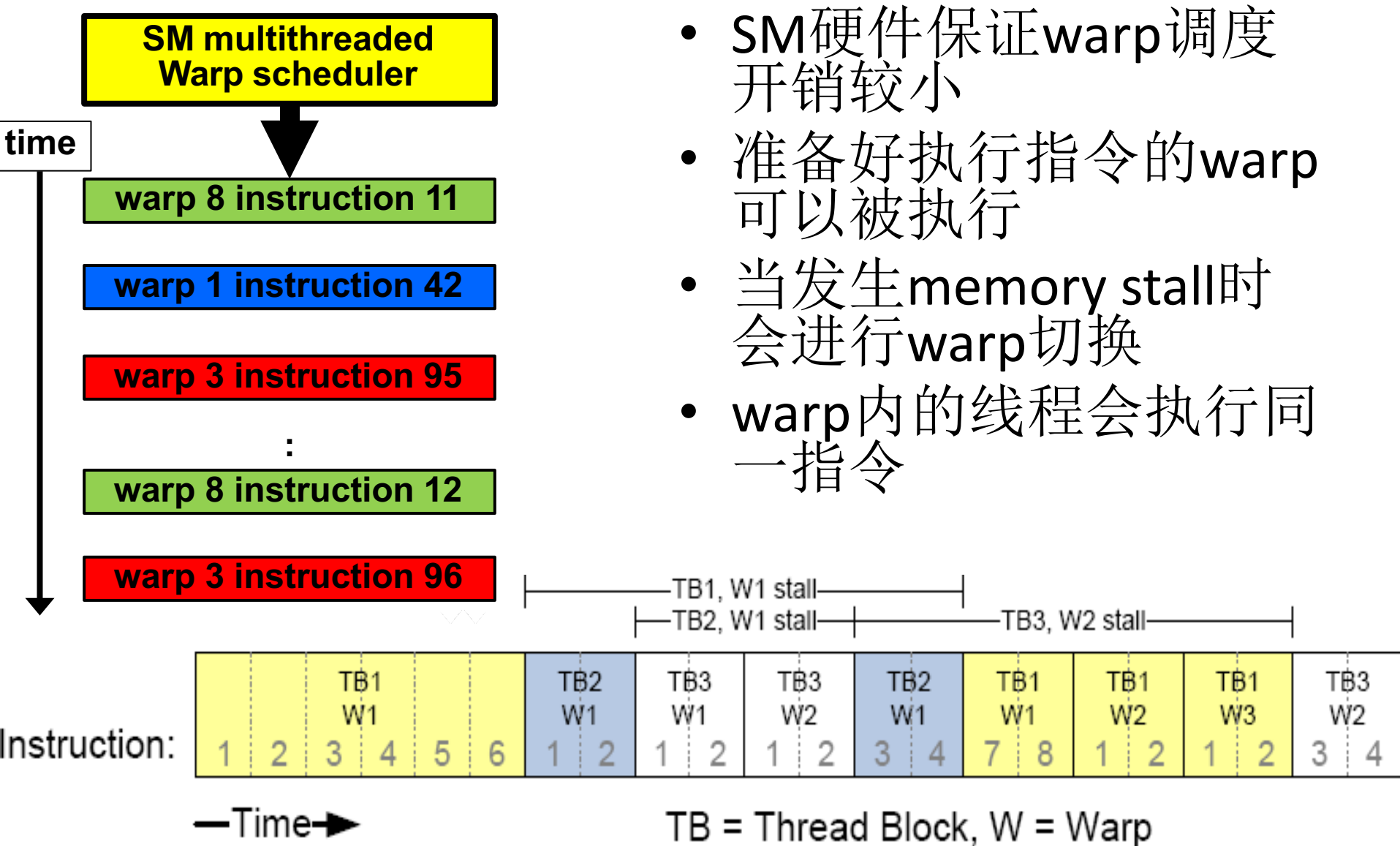
warp

- SM内部，线程以warp为单位，32个线程一组进行发射
 - warp共享控制单元（warp scheduler）
 - 每个warp scheduler同一时间只有一个warp在SM执行
 - 同一个warp内的线程执行同样的语句（SIMD）
- 换句话说。。。
 - warp内的线程物理上并行执行
 - warps和blocks逻辑上并行执行
 - block内的线程需要同步操作





warp调度器warp scheduler



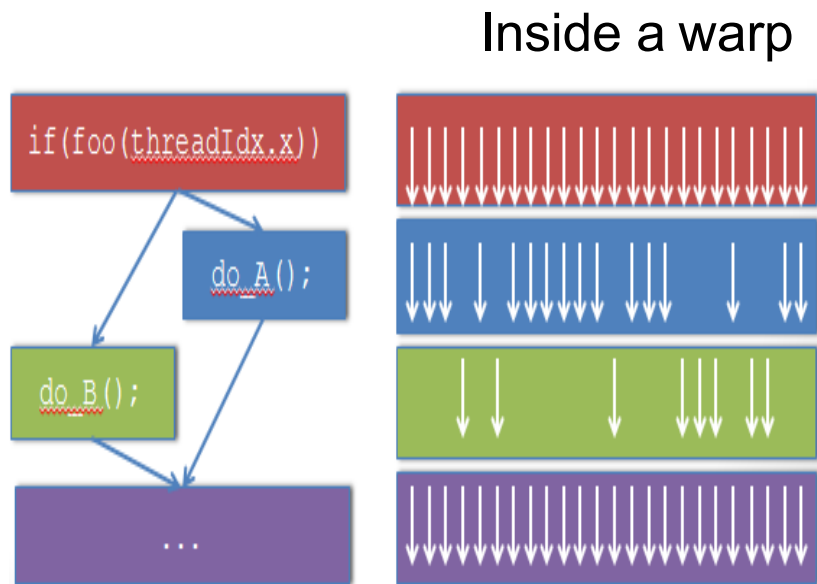


warp divergence

- 如果同一个warp内的不同线程在做不同的事情，会发生什么？
 - flow control instruction (if, switch, do, for, while)

```
if(foo(threadIdx.x)) { do_A(  
    );  
} else {  
    do_B();  
}
```

- warp内的不同执行路径被串行化
 - 潜在性能开销很大





如何避免warp diverging

- warp divergence例子

- ```
if (threadIdx.x > 2) {...}
else {...}
```

- Branch granularity < warp size

- 消除divergence的例子

- ```
if (threadIdx.x / WARP_SIZE > 2) {...}
else {...}
```

不同的warp执行不同的代码，对性能无影响
branch granularity是warp size的整数倍数



Example: Divergent Iteration

```
__global void per_thread_sum
(int *indices, float *data, float *sums){
    ...
    // number of loop iterations is
    // data dependent
    int i = threadIdx.x
    for(int j=indices[i]; j<indices[i+1]; j++) {
        sum += data[j];
    }
    sums[i] = sum;
}
```



Iteration Divergence

- 单个线程可能拖慢整个warp的执行时间，因为其他线程要等单个线程执行
- 程序员需要了解数据分布模式
- 如果数据不可预测，可以尝试使用一种扁平的分布，让每个线程作用于多个数据项
 - BFS
 - SpMV



for-loop unroll

- 循环展开有助于消除branches，增加 pipeline

■ Example:

- ```
for (i=0; i<n; i++)
{
 a = a + i;
}
```
- Unrolled 3 times
- ```
for (i=0; i<n; i+=3)  
{  
    a = a + i;  
    a = a + i+1;  
    a = a + i+2;  
}
```



#pragma unroll

- #pragma unroll 指导语句可以用来展开loop
- 必须放在需要展开的loop前
- 例子
 - #pragma unroll 5
 - for (int i = 0; i < n; ++i)
 - 循环将被展开 **unroll 5 次**
 - 编译器还会插入其他代码保证正确性
 - #pragma unroll 1 防止编译器进行循环展开



原子操作

- 在有些情况下，线程可能需要更新 shared/global memory 中的数值
 - `_shared int count;`
 - `.....`
 - `if (.....) count++;`
- 一致性问题：如果有2个或多个线程都执行同样的代码
- 解决：使用GPU支持的原子操作
 - `addition / subtraction`
 - `max / min`
 - `increment / decrement`
 - `compare-and-swap`



Example: Histogram

```
/* Determine frequency of colors in a picture  
colors have already been converted into ints. Each  
thread looks at one pixel and increments a counter  
atomically*/
```

```
__global void hist(int* color, int* bin){  
    int i = threadIdx.x + blockDim.x *  
            blockIdx.x;  
  
    int c = colors[i];  
    atomicAdd(&bin[c], 1);  
}
```



Example: Global Min/Max

```
__global void global_max(int* values,  
int* gl_max) {  
    int i = threadIdx.x + blockDim.x *  
            blockDim.x;  
  
    int val = values[i];  
    atomicMax(gl_max, val);  
}
```

- Not very fast for data in shared memory
- Only slightly slower for data in device memory



思考

- Warp是如何执行的？



目录

- CUDA内容总结
- 线程执行模型
- 存储层次结构
 - 寄存器 / local memory
 - shared memory
 - global/constant memory

GPU Memory Hierarchy

■ Registers

- Read/write per-thread
- Low latency & High BW

■ Shared memory

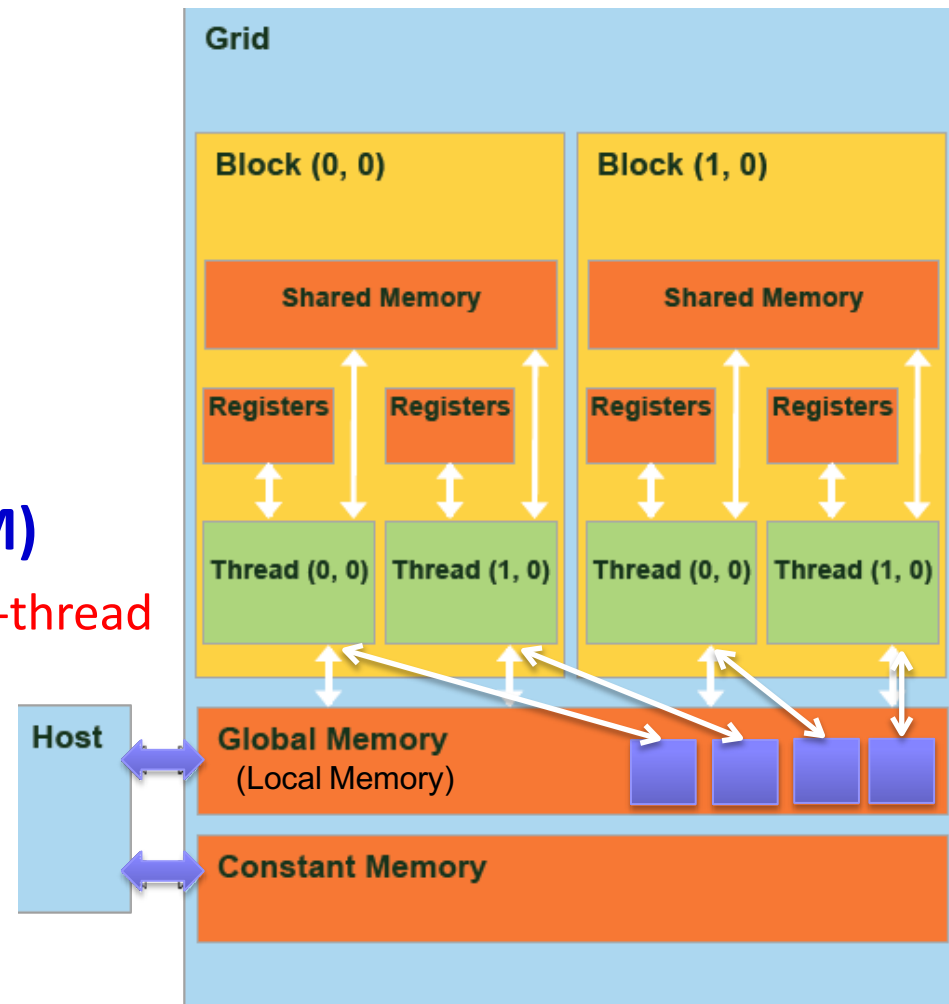
- Read/write per-block
- Similar to register performance

■ Global/Local memory (DRAM)

- Global is per-grid & Local is per-thread
- High latency & Low BW
- Not cached

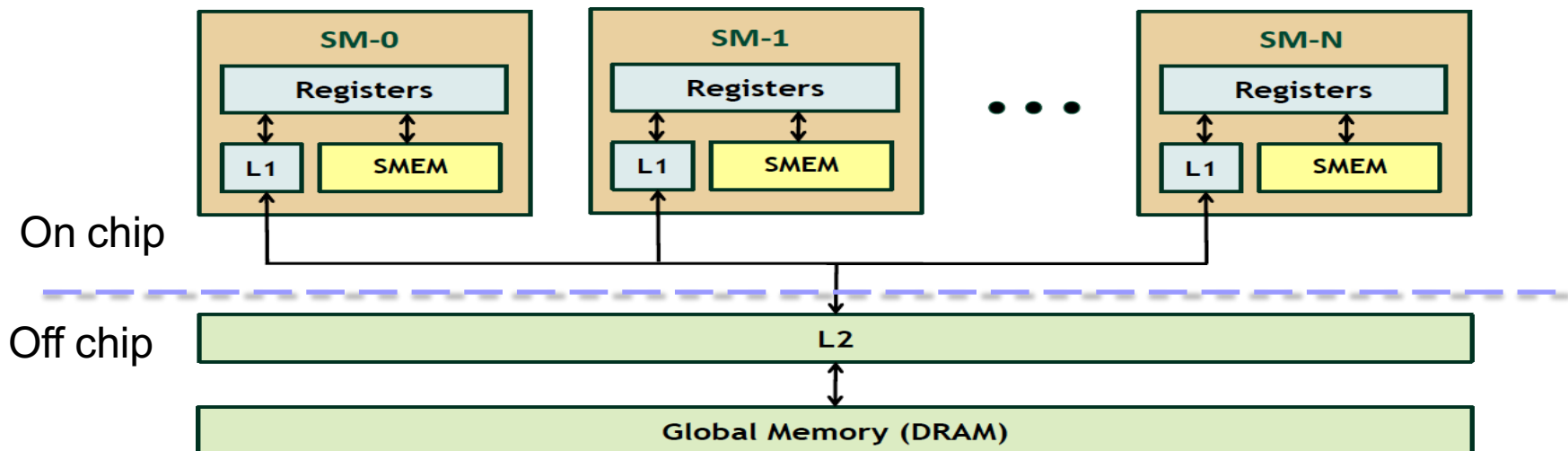
■ Constant memory

- Read only per-grid
- Cached



内存访问

- **store操作向L1写数据 A store writes a line to L1**
 - If evicted, that line is written to L2
 - The line could also be evicted from L2 and written to DRAM (global mem.)
- **load操作从L1读数据 A load requests the line from L1**
 - If a hit, operation is complete
 - If a miss, then requests the line from L2
 - ◆ If a miss, then requests the line from DRAM (global memory)
- **Only GPU threads can access local memory addresses**





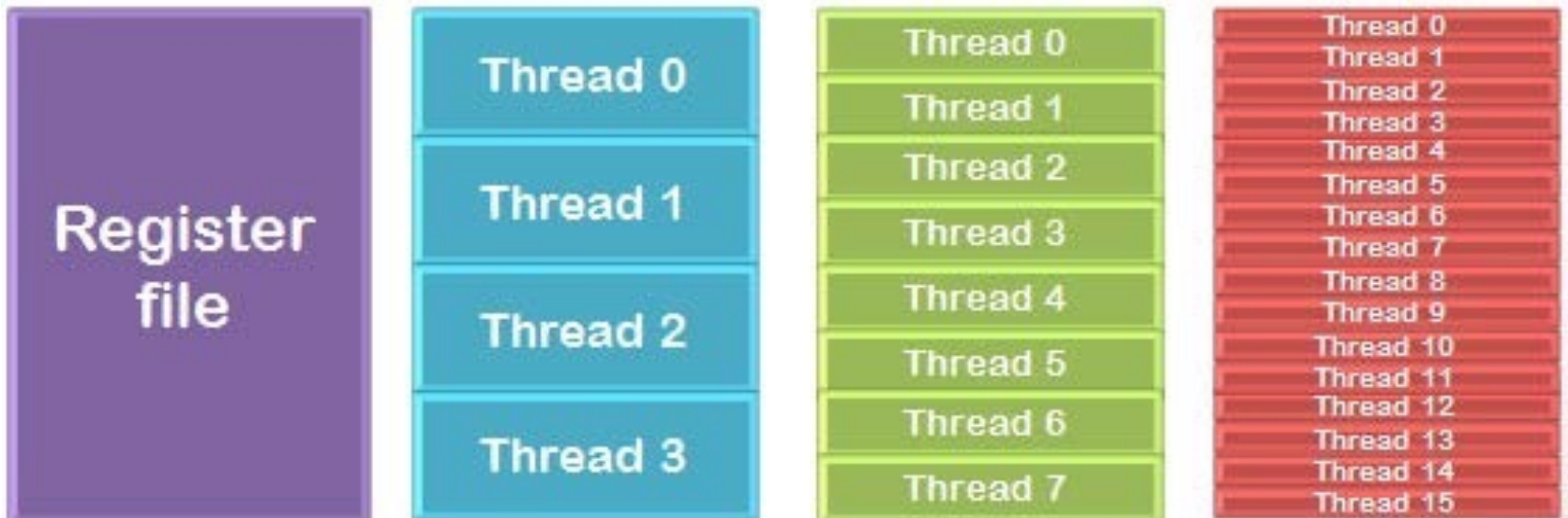
寄存器

- 寄存器操作几乎没有带来时钟周期上的开销，但有如下例外情况
 - read-after-write寄存器依赖
 - register memory bank conflicts
- 寄存器溢出register spilling
 - 每个线程最大的寄存器数量有限
 - 如果超出限制会使用local memory
 - array总是使用local memory (DRAM)



register pressure

- 线程数目太少
 - 无法隐藏pipeline/memory access latency
- 线程数太多
 - 寄存器有压力
 - 线程分配到的寄存器数目有限
 - 每个活跃的block分到的shared memory有限





local memory

- 指寄存器溢出或其他各线程使用的内存资源
 - 通常由于SM资源用尽
 - local的原因是各线程私有
- 并不是独立的memory, bytes存储于global memory(DRAM)
- 和global memory区别在于:
 - 寄存器分配
 - store操作缓存于L1



例子

```
__device void distance(int m, int n, int *V) {  
    int i, j, k;  
    int a[10], b[10], c[10];  
    ...  
}
```

- Variables *i, j, k, a, b, c* 称为 “local variables”.
- 很可能variable *i, j, k* 存储于寄存器, 同时 variable *a, b, c* 存储于“local memory” (off-chip DRAM).
 - Compiler 编译器决定存放位置
 - Registers不可编号索引, 所以数组通常存储于local memory.
 - 如果没有足够的寄存器, 就会使用local memory.



目录

- CUDA内容总结
- 线程执行模型
- 存储层次结构
 - 寄存器 / local memory
 - **shared memory**
 - global/constant memory



shared memory

- 可以编程控制的cache
 - 几乎和寄存器一样快
- 范围：block内的所有线程
 - block内的线程可以通过shared memory进行通信
 - 不同block内的线程只能通过global memory进行通信
- 大小：每代不一样
 - Fermi/Kepler GPU，shared memory和L1 cache使用相同的存储硬件（64K），程序员决定shared memory和L1 cache的比例
 - 3:1/1:1/1:3



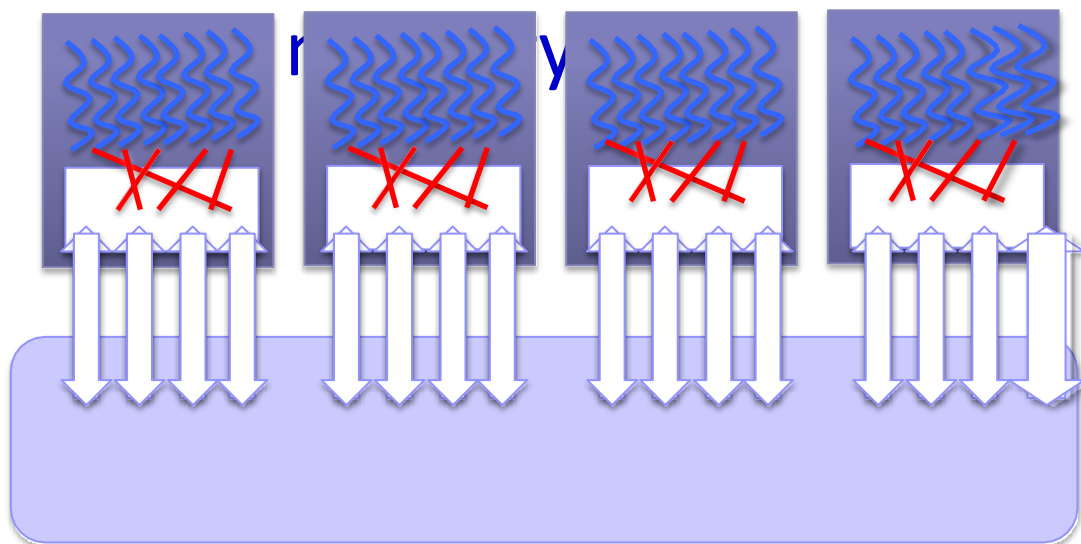
shared memory限制

Table 15. Technical Specifications per Compute Capability

Table 10: Technical specifications per compute capability													
Technical Specifications	3.5	3.7	5.0	5.2	5.3	6.0	6.1	6.2	7.0	7.2	7.5	8.0	8.6
Maximum number of resident grids per device (Concurrent Kernel Execution)	32				16	128	32	16	128	16	128		
Maximum dimensionality of grid of thread blocks	3												
Maximum x-dimension of a grid of thread blocks	2 ³¹ -1												
Maximum y- or z-dimension of a grid of thread blocks	65535												
Maximum dimensionality of a thread block	3												
Maximum x- or y-dimension of a block	1024												
Maximum z-dimension of a block	64												
Maximum number of threads per block	1024												
Warp size	32												
Maximum number of resident blocks per SM	16		32								16	32	16
Maximum number of resident warps per SM	64										32	64	48
Maximum number of resident threads per SM	2048										1024	2048	1536
Number of 32-bit registers per SM	64 K	128 K	64 K										
Maximum number of 32-bit registers per thread block	64 K				32 K	64 K			32 K	64 K			
Maximum number of 32-bit registers per thread	255												
Maximum amount of shared memory per SM	48 KB	112 KB	64 KB	96 KB	64 KB		96 KB	64 KB	96 KB		64 KB	164 KB	100 KB
Maximum amount of shared memory per thread block ³³	48 KB								96 KB	96 KB	64 KB	163 KB	99 KB
Number of shared memory banks	32												
Maximum amount of local memory per thread	512 KB												
Constant memory size	64 KB												
Cache working set per SM for constant memory	8 KB				4 KB	8 KB							
Cache working set per SM for texture memory	Between 12 KB and 48 KB				Between 24 KB and 48 KB				32 - 128 KB		32 or 64 KB	28KB - 192 KB	28KB - 128 KB
Maximum width for a 1D texture reference bound to a CUDA array	65536				131072								
Maximum width for a 1D texture reference bound to linear memory	2 ²⁷				2 ²⁸	2 ²⁷			2 ²⁸	2 ²⁷	2 ²⁸		
Maximum width and number of layers for a 1D layered texture reference	16384 x 2048				32768 x 2048								
Maximum width and height for a 2D texture reference bound to a CUDA array	65536 x 65536				131072 x 65536								
Maximum width and height for a 2D texture reference bound to linear memory	65000 x 65000		65536 x 65536			131072 x 65000							
Maximum width and height for a 2D texture reference bound to a linear memory	16384 x 16384				32768 x 32768								

总体策略

- 从global memory往shared memory中存数据
- 在shared memory中处理数据
- 将shared memory写回global memory



Blocks

Shared memory

Global memory

举例

- Use **$n*n$ threads**.
- Each updates the shortest path of one pair vertices
- Use **global memory** to store the matrix D.

```
__global void FW_APSP(int k, int D[n][n]) {  
    int i = threadIdx.x;  
    int j = threadIdx.y;  
    if (D[i][j] > D[i][k] + D[k][j])  
        D[i][j] = D[i][k] + D[k][j];  
}  
int main() { ...  
    dim3 threadsPerBlock(n, n);  
    for (int k = 0; k < n; k++)  
        FW_APSP<<<1, threadsPerBlock >>>(k, D);  
}
```

6 global
memory
access



使用Shared Memory

- This way of using shared memory is called **dynamic allocation of shared memory**, whose size is specified in the kernel launcher.

```
FW_APSP<<<1, n*n, n*n*sizeof(int)>>> (...);
```

- The third parameter is the size of shared memory.

```
extern__shared int S[][];  
__global void FW_APSP(int k, int D[n][n]) {  
    int i = threadIdx.x;  
    int j = threadIdx.y;  
    S[i][j]=D[i][j]; // move data to shared memory  
    __syncthreads();  
    // do computation  
    if (S[i][j]>S[i][k]+S[k][j])  
        D[i][j]= S[i][k]+S[k][j];  
}
```

ONLY 2
global mem
access



static shared memory allocation

- 如果shared memory在编译阶段能够知道大小，就可以静态进行分配

```
_global void FW_APSP(int k, int D[][]){  
    shared int DS[10*10];  
    ...  
}
```

Must know
n=10 at
compile time



目录

- CUDA内容总结
- 线程执行模型
- 存储层次结构
 - 寄存器 / local memory
 - shared memory
 - global/constant memory



kernel的global memory

- 通过cudaMalloc分配内存，通过cudaMemcpy设置内容
- 作业中用到
- `cudaMemcpy( void *dst, const void`
- `*src, size_t count, enum`
- `cudaMemcpyKind kind)`



constant memory

- 使用方法和范围与global memory一样，除了以下情况
- 只读
 - 使用关键字 `__constant`
 - Ex: `__constant int data[32];`
- 每个SM有自己独立的constant memory
 - 对于Fermi，每个SM的constant memory为64K，有独立的cache, 8K



CUDA kernel内的变量

Variable declaration	Memory	Scope	Lifetime
<code>int var</code>	Register	Thread	Thread
<code>int array_var[10]</code>	Local	Thread	Thread
<code>__shared__ int shared_var</code>	Shared	Block	Block
<code>__device__ int global_var</code>	Global	Grid	App
<code>__constant__ int constant_var</code>	Constant	Grid	App

- scalar变量如果没有qualifier就会分配在寄存器中
- 有可能编译时候溢出到local memory中
- 数据如果没有qualifier通常分配到thread私有local memory



存储速度

Variable declaration	Memory	Speed
<code>int var</code>	Register	1x
<code>int array_var[10]</code>	Local	100x
<code>__shared__ int shared_var</code>	Shared	1x
<code>__device__ int global_var</code>	Global	100x
<code>__constant__ int constant_var</code>	Constant	1x

- scalar变量存储于快速片上寄存器
- shared variable存储于快速的片上内存
- thread-local数据 / 全局变量存储于 uncached-off chip memory
- constant 变量存储于cached off-chip memory



思考

- 存储层次结构的区别
 - 寄存器 / local memory
 - shared memory
 - global/constant memory



一些CUDA程序的例子



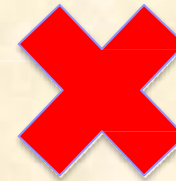
Example 1: Hello World!

```
__global__ void mykernel(void) {  
  
    int main(void)  
    { mykernel<<<1,1>>>();  
      printf("Hello World!\n");  
      return 0;  
    }  
}
```

- `__global__` 表示函数运行在GPU，由CPU调用
- `mykernel<<<1,1>>>` 表示 **kernel launch**

Example 2: add 2 numbers

```
__global void add(int *a, int *b, int *c) {  
    *c = *a + *b;  
}  
  
int main(void) {  
    int ha=1,hb=2,hc;  
    add<<<1,1>>>(&ha, &hb, &hc);  
    printf("c=%d\n",hc);  
    return 0;  
}
```



- 不正确
- ha, hb, hc属于host memory, 不能运行在GPU上
- 需要在device memory上分配变量



正确的 main ()

```
int main(void) {  
    int a=1, b=2, c; // host copies of a, b, c  
    int *d_a, *d_b, *d_c; // device copies of a, b, c  
    // Allocate space for device copies of a, b, c  
    cudaMalloc((void **)&d_a, sizeof(int));  
    cudaMalloc((void **)&d_b, sizeof(int));  
    cudaMalloc((void **)&d_c, sizeof(int));  
    // Copy inputs to device  
    cudaMemcpy(d_a, &a, sizeof(int), cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, &b, sizeof(int), cudaMemcpyHostToDevice);  
    // Launch add() kernel on GPU  
    add<<<1,1>>>(d_a, d_b, d_c);  
    // Copy result back to host  
    cudaMemcpy(&c, d_c, size, cudaMemcpyDeviceToHost);  
    // Cleanup  
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);  
    return 0;  
}
```



Example 3: add 2 vectors

- 串行版本

```
// function definition
void VecAdd(int N, float* A, float* B, float* C)
{
    for(int i = 0; i<N; i++)
        C[i] = A[i] + B[i];
}

int main()
{ ...
    VecAdd (N, Ah, Bh, Ch);
    ...
}
```



并行CUDA版本v1

- 1个block, 多个threads

```
// Kernel definition
__global void VecAdd(float* A, float* B, float* C)
{
    int i = threadIdx.x;
    C[i] = A[i] + B[i];
}

int main()
{ ...
    // Kernel invocation with N threads
    VecAdd<<<1, N>>>(Ah, Bh, Ch); ...
}
```



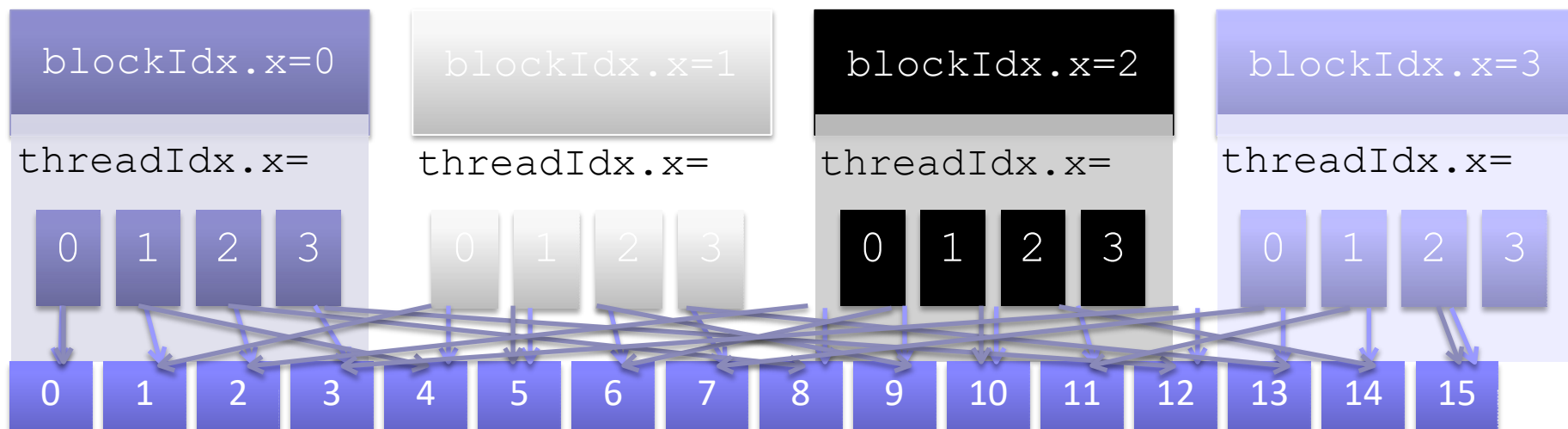
并行CUDA版本v2

- 多个block，每个block1个线程

```
_global void add(int *a, int *b, int *c) {  
    c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];  
}  
  
int main(void) {  
    int a[N], b[N],  
    c[N];  int *d_a,  
    *d_b, *d_c;  
    ...  
    add<<< N, 1 >>>(d_a, d_b, d_c);  
    ...  
}
```

并行CUDA版本v3

- 使用多个线程，多个block
- 假设 $N=16$, grid size=4, block size=4



➤ 哪种索引方式好?

➤ Method 1: $\text{index} = \text{blockIdx.x} * 4 + \text{threadIdx.x}$

➤ Method 2: $\text{index} = \text{threadIdx.x} * 4 + \text{blockIdx.x}$



并行CUDA版本v3

- 使用blockDim.x

```
__global void add(int *a, int *b, int *c) {  
    int index = threadIdx.x + blockIdx.x * blockDim.x;  
    c[index] = a[index] + b[index];  
}  
  
int main(void) {  
    int a[N], b[N], c[N];  
    int *d_a, *d_b, *d_c;  
    ...  
    add<<< N/BS, BS>>>>(d_a, d_b, d_c);  
    ...  
}
```

What if N is not a multiple of BS?

■ BS is block size (number of threads per block)



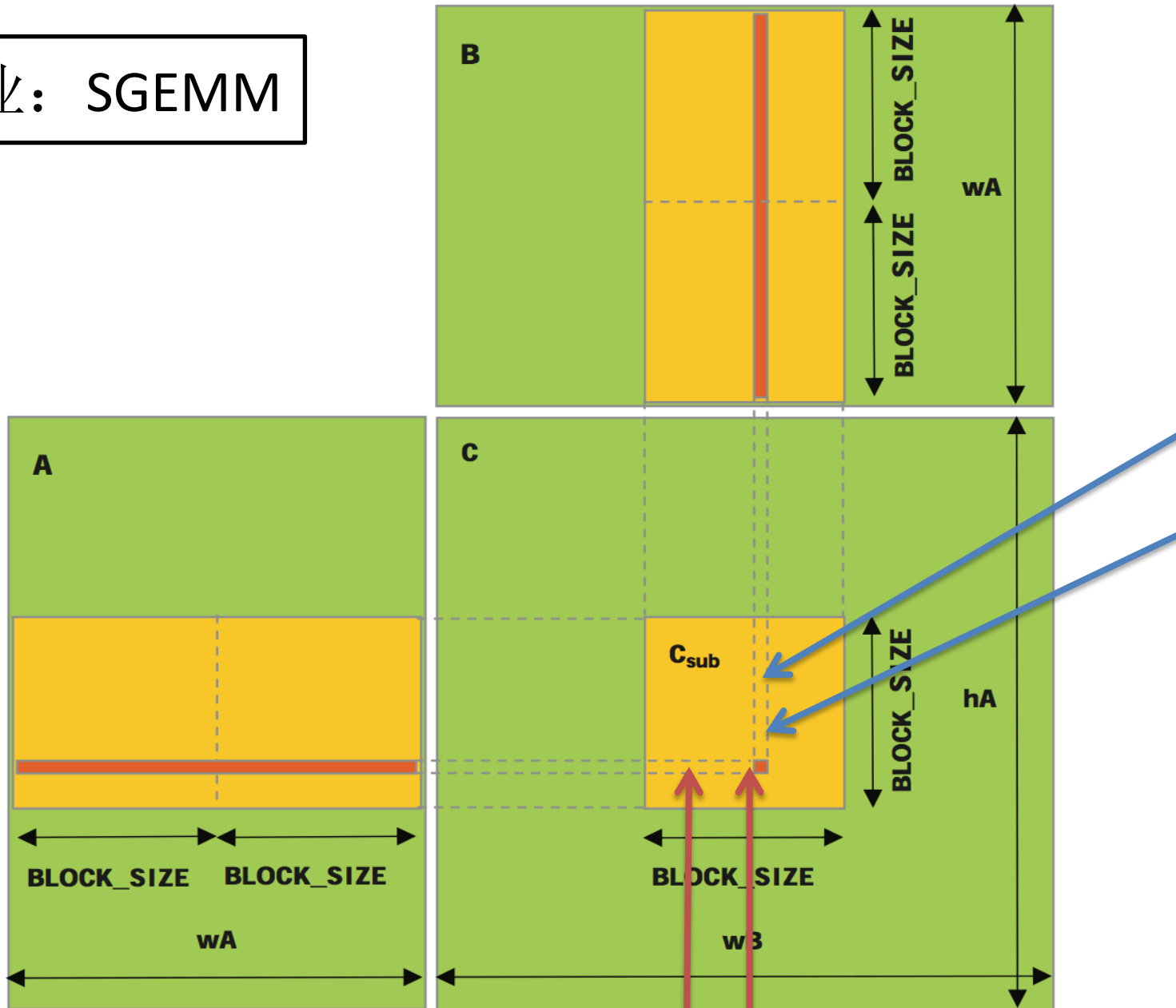
并行CUDA版本v3

```
__global void add(int *a, int *b, int *c, int n) {  
    int index = threadIdx.x + blockIdx.x * blockDim.x;  
    if (index < n)  
        c[index] = a[index] + b[index];  
}  
  
int main(void) {  
    int a[N], b[N], c[N];  
    int *d_a, *d_b, *d_c;  
    ...  
    add<<< (N+BS-1)/BS, BS>>>(d_a, d_b, d_c, N);  
    ...  
}
```

- branches带来一些性能开销



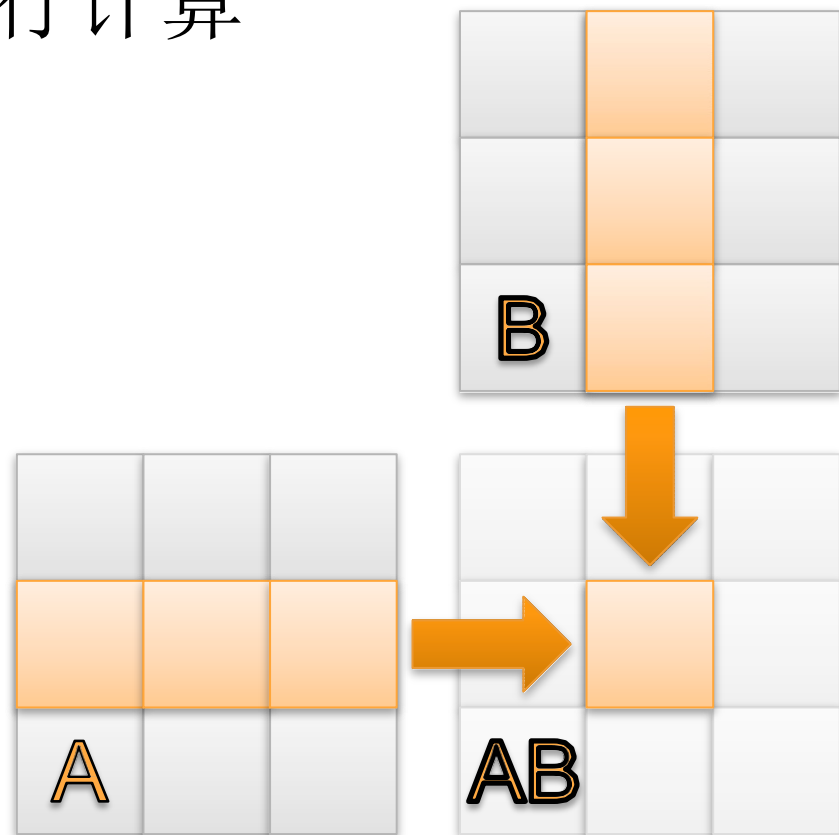
作业: SGEMM



Each thread block computes one sub-matrix C_{sub} of C. Each thread within the block computes one element of C_{sub} .

矩阵矩阵乘法

- $AB = A * B$
- 每个 Ab_{ij} 由 1 个线程进行计算
- $\text{dot}(\text{row}(A,i), \text{col}(B,j))$





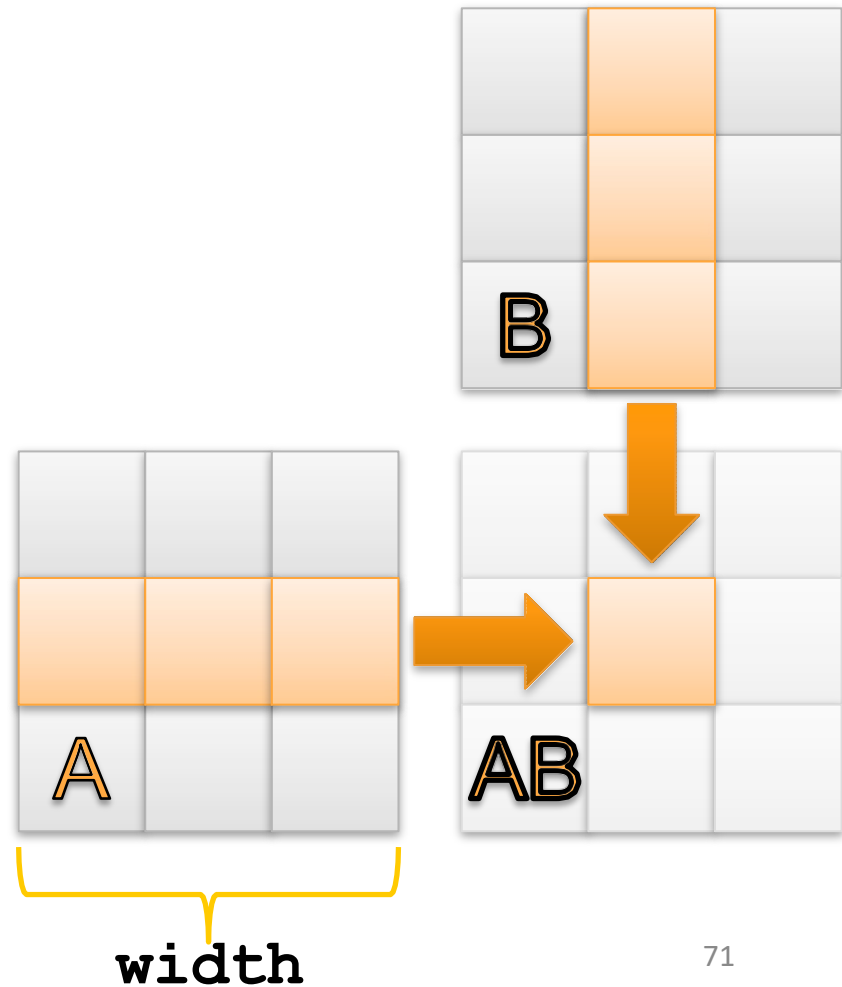
版本1

```
__global void mat_mul(float *a, float *b,  
                      float *ab, int width)  
{  
    // calculate the row & col index of the element  
    int row = blockIdx.y*blockDim.y + threadIdx.y;  
    int col = blockIdx.x*blockDim.x + threadIdx.x;  
  
    float result = 0;  
  
    // do dot product between row of a and col of b  
    for(int k = 0; k < width; ++k)  
        result += a[row*width+k] * b[k*width+col];  
  
    ab[row*width+col] = result;  
}
```

GTX260
28GFLOPS

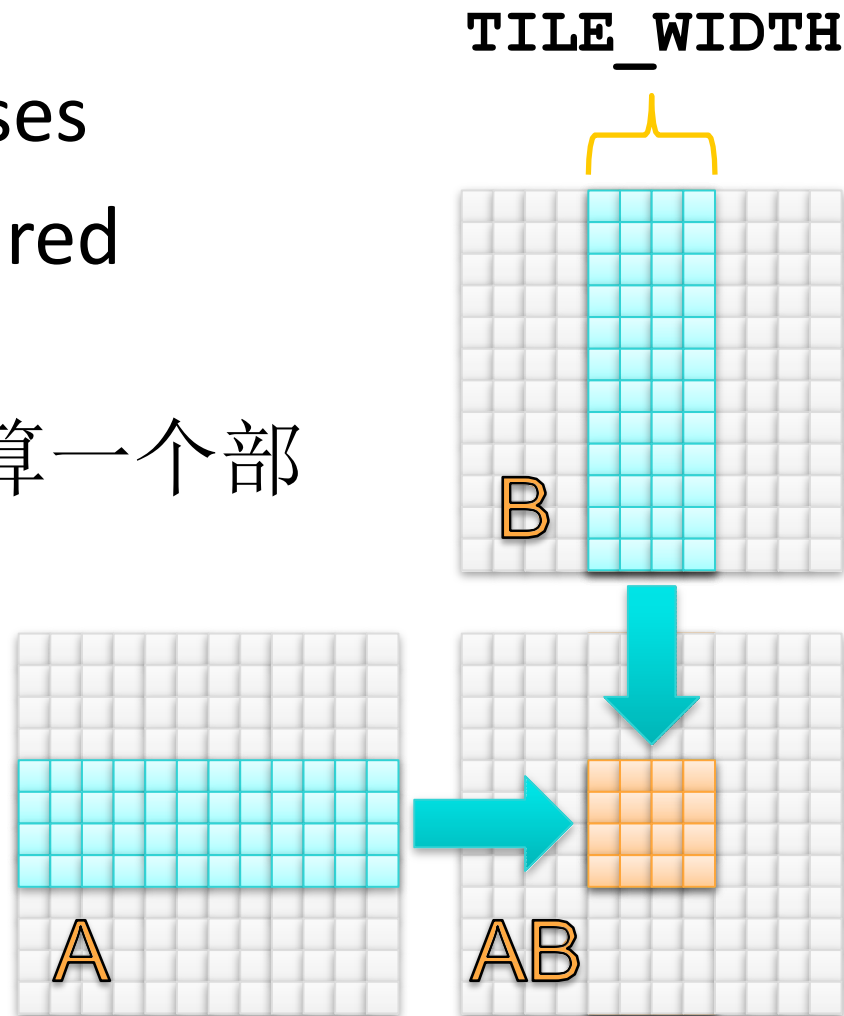
版本2

- 使用__shared__ memory 存储global data
- input的数据被线程缓存在local memory
- 之后block内的线程访问local memory降低带宽使用情况



Tiled Multiply

- 将kernel loop划分为phases
- 将划分后矩阵存储于shared memory
- 每个phase, 各线程只计算一个部分结果 partial result





版本2

```
__global__ void mat_mul(float *a, float *b,  
                        float *ab, int width)  
{  
    // shorthand  
    int tx = threadIdx.x, ty = threadIdx.y;  
    int bx = blockIdx.x, by = blockIdx.y;  
    // allocate tiles in shared memory  
    shared float s_a[TILE_WIDTH][TILE_WIDTH];  
    shared float s_b[TILE_WIDTH][TILE_WIDTH];  
    // calculate the row & col index  
    int row = by*blockDim.y + ty;  
    int col = bx*blockDim.x + tx;  
  
    float result = 0;
```



```
// loop over the tiles of the input in phases
for(int p = 0; p < width/TILE_WIDTH; ++p)
{
    // collaboratively load tiles into shared
    s_a[ty][tx] = a[row*width + (p*TILE_WIDTH + tx)];
    s_b[ty][tx] = b[(m*TILE_WIDTH + ty)*width + col];
    __syncthreads();

    // dot product between row of s_a and col of s_b
    for(int k = 0; k < TILE_WIDTH; ++k)
        result += s_a[ty][k] * s_b[k][tx];
    __syncthreads();
}

ab[row*width+col] = result;
}
```



barrier操作

- 每个phase会出现2次barrier
 - 存入数据到shared memory后需要一次__syncthreads
 - 计算结果读出数据后需要一次__syncthreads
 - 注意第二次__syncthreads是为了保证下一次存入数据的正确性
- 使用barrier维护数据：
 - 被初始化
 - 读正确的数据

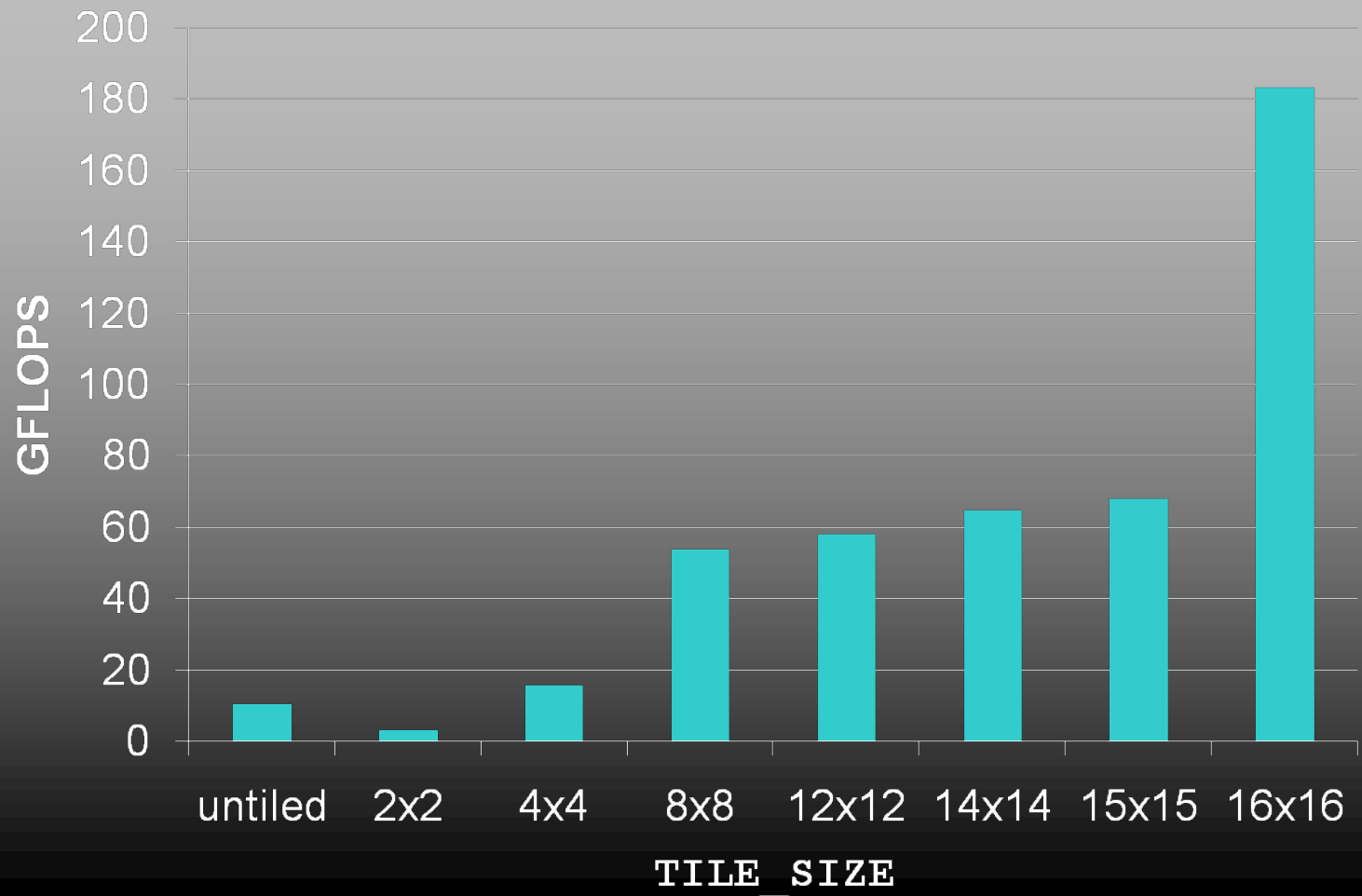


分块大小考虑因素

- 每个thread block有许多线程
- $\text{TILE_WIDTH} = 16 * 16 = 256$ threads
- 有很多的thread blocks
- $1024 * 1024$ 的矩阵 $\rightarrow 64 * 64 = 4096$ 个thread block



TILE_SIZE 影响



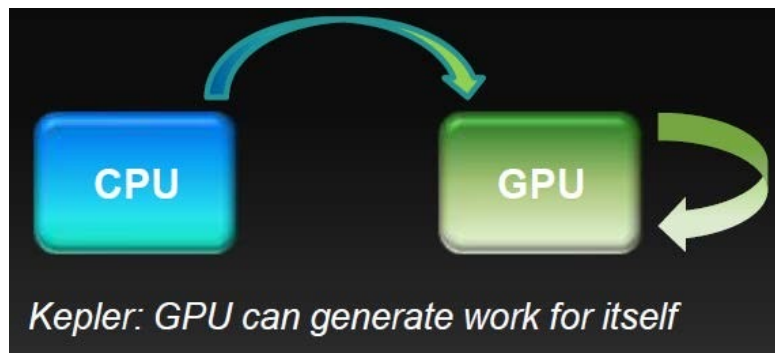
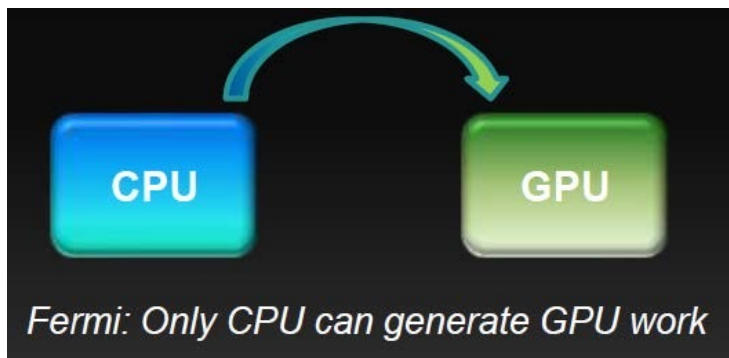


优化因素

- 合理使用CUDA内存降低带宽使用情况，增加throughput
- 使用shared memory增加对global memory的重用情况
 - __syncthreads操作保证共享数据的初始化
 - 使用原子操作避免写冲突
- 各优化操作也会有性能开销
- 内存资源的使用情况会限制并行程度

动态并行dynamic parallelism

- 能从GPU内部开启新的grid
 - 动态Dynamically
 - 同时Simultaneously
 - 独立Independently
- CUDA5.0后支持Compute Capability 3.5 或更高版本

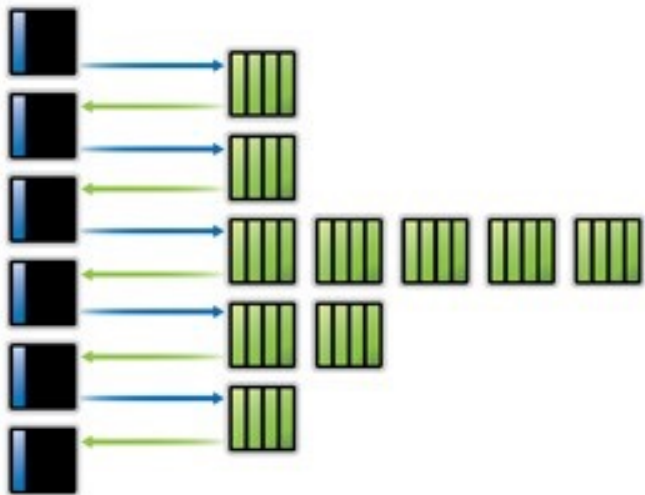


动态并行意义

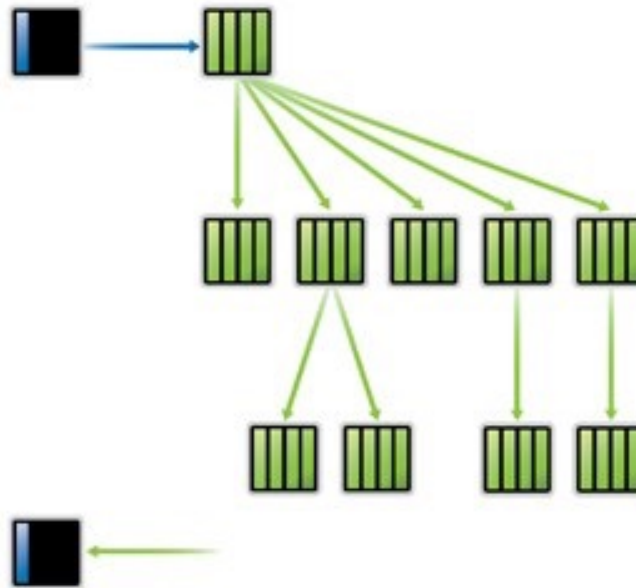
- 减少kernel launch的次数

DYNAMIC PARALLELISM

CPU GPU



CPU GPU





动态并行例子

```
__global__ ChildKernel(void* data){  
    //Operate on data  
}  
  
__global__ ParentKernel(void *data){  
    ChildKernel<<<16, 1>>>(data);  
}  
  
// In Host Code  
ParentKernel<<<256, 64>>(data);
```

- 支持递归调用

```
__global__ RecursiveKernel(void* data){  
    if(continueRecursion == true)  
        RecursiveKernel<<<64, 16>>>(data);  
}
```

谢谢！



中國人民大學
RENMIN UNIVERSITY OF CHINA